

ROS

ROS is an open-source meta-operating system used for robots. It provides an operating system with expected services, including hardware abstraction, low-level device control, implementation of common functions, messages transferred between processes, and package management. It also provides the tools and library functions needed to obtain, compile, write, and run codes across computers.

The "graph" of ROS runtime is a loosely coupled peer-to-peer process network based on a ROS communication infrastructure. ROS implements several different communication methods, including a services mechanism based on synchronous RPC-style communication, a topics mechanism based on asynchronous streaming media data, and a parameter server for data storage.

ROS is not a real-time framework, but it can be embedded in real-time programs. Willow Garage's PR2 robot uses a system called pr2_etherCAT to send or receive ROS messages in real time. ROS may also be seamlessly integrated with Orocos real-time toolkits.

ROS Logo :



1 Design goals and characteristics of ROS

Many people ask "what are differences between ROS and other robot software platforms?" This question is difficult to answer. Because ROS is not a framework that integrates most functions or features. In fact, the main goal of ROS is to provide code reuse support for robot R&D. ROS is a framework (i.e. nodes) for distributed processes, which are encapsulated in program and function packages that can be easily shared and distributed. ROS also supports a federated system similar to a code repository, and this system also enables the collaboration and distribution of a project. This design enables the development and realization of a project to be decided completely independently from the file system to the user interface (no limit by ROS). At the same time, all projects can be integrated with the basic tools of ROS.

To support the primary goals of sharing and collaboration, the ROS framework has several other features:

- **Lean:** ROS is designed to be as lean as possible so that codes written for ROS can be used with other robot software frameworks. The inevitable conclusion from this is that ROS can be easily integrated into other robot software platforms: ROS can already be integrated with OpenRAVE, Orocos and Player.
- **ROS insensitive libraries:** The preferred development model of ROS is written with clean library functions that do not depend on ROS.
- **Language independence:** The ROS framework can simply be implemented in any modern programming language. ros has implemented Python version, C++ version and Lisp version. It also has experimental libraries for Java and Lua versions.
- **Loose coupling:** The function modules in ROS are encapsulated in independent function packages or meta-function packages, which are easy to share. The modules in the function package are run in units of nodes. With ROS standard IO used as the interface, developers does not need to pay attention to the internal implementation of the module, as long as they

understand the interface rules, they can achieve reuse and point-to-point loose coupling between modules.

- Convenient testing: ROS has a built-in unit/integration testing framework called rostest, which can easily install or uninstall test modules.
- Scalable: ROS is applicable to large runtime systems and large development processes.
- Free and open-source: It has many developers and many function packages.

2 Why ROS is used?

Through ROS, we can realize the simulated control of robot arms in a virtual environment.

we will visualize robot arms through **rviz**, operates our robot arms in a variety of ways, and plan and executes the action path of robot arms through **MoveIt** to achieve the effect of freely controlling the robot arms.

We will learn how to control our products through the platform in ros in the following chapters.

MoveIt

MoveIt is currently the most advanced software for movement operations of robot arms and has been used for more than 100 robots. It integrates the latest achievements in motion planning, control, 3D perception, motion control, control and navigation, provides an easy-to-use platform for developing advanced robot applications, and provides an integrated software platform for design, and integration assessment of new robot products in the fields of industry, commerce, R&D, etc.

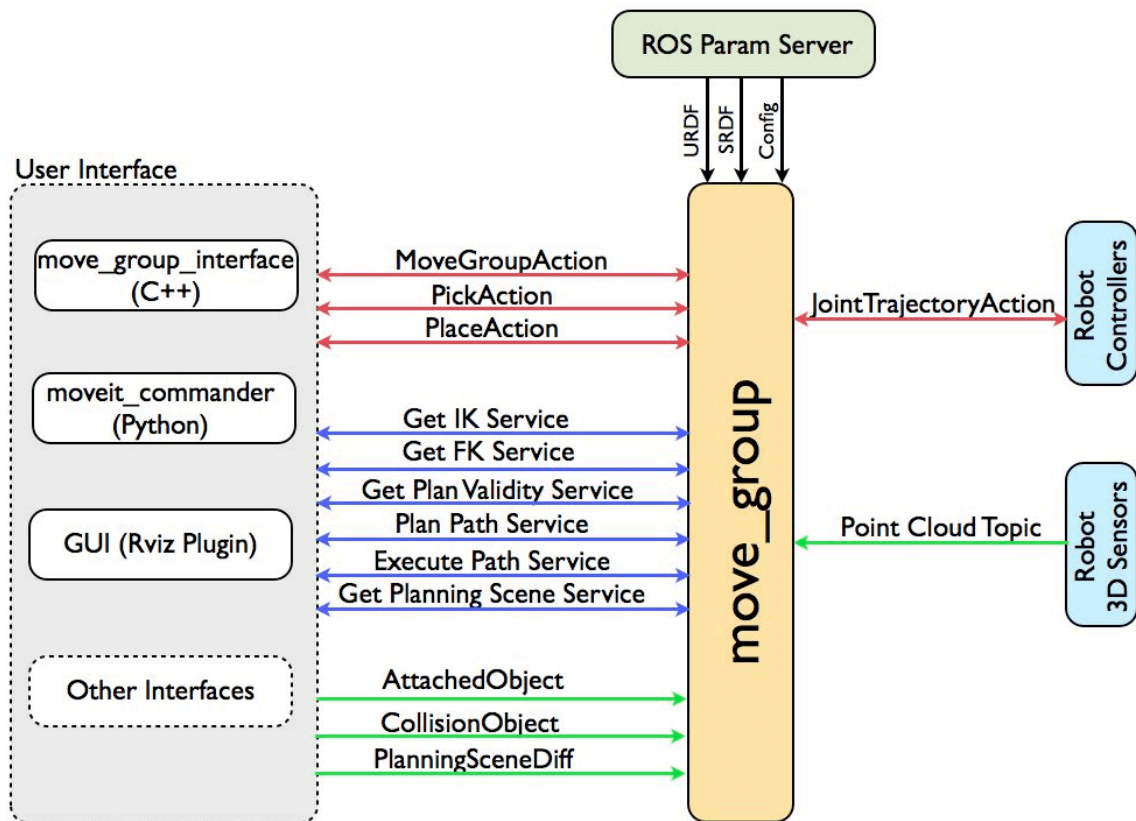
MoveIt Logo :



1 Introduction

MoveIt is an integrated development platform in ROS, which consists of a variety of functional packages for manipulating robot arms, including motion planning, operation, control, inverse kinematics, 3D perception, collision detection, etc.

The following figure shows the high-level structure of the main node **move_group** provided by Moveit. It is like a combiner: all the individual components are integrated together, providing a series of actions and services for users to use.



2 User interface

The user may access the operations and services provided by move_group in three ways:

- In C++, you may use move_group easily by using move_group_interface package.
- In Python, use the moveit_commander package.
- Via GUI: use Rviz (ROS visualization tool) of Motion-commander.

move_group can be configured using the ROS parameter server, from which the robot's URDF and SRDF can also be obtained.

3 Configuration

move_group is a ROS node. It uses the ROS parameter server to obtain three kinds of information:

- URDF - move_group looks for the robot_description parameter in the ROS parameter server to get the robot's URDF.
- SRDF - move_group looks for the robot_description_semantic parameter in the ROS parameter server to get the robot's SRDF. SRDF is typically created by the user using an Movelt Setup Assistant.
- Movelt configuration - move_group will look in the ROS parameter server for additional Movelt-specific configurations, including joint constraint, kinematics, motion planning, perception, and other information. The configuration files for these components are automatically generated by the Movelt Setup Assistant and stored in the configuration directory of the robot's corresponding Movelt configuration package. For the use of configuration assistant, please refer to: [Movelt Setup Assistant](#)